

STORYWORLDS: Programmable Narrative Worlds for Reproducible Agent Evaluation and Data Generation

Patrick Dugan

February 28, 2026, updated March 2

Abstract

We introduce STORYWORLDS, a framework for constructing *programmable narrative worlds*—formalized environments in which language-model agents interact through structured state, action affordances, and enforceable outcome constraints. STORYWORLDS unifies (i) evaluation of agentic competence under long-horizon pressure, (ii) trace generation for instruction-tuning and adapters, and (iii) safety-oriented testing of goal pursuit under narrative temptation, for constitutional and other alignment training. A storyworld defines a latent world state of a set of variables, a partial observability interface, a series of encounters encoding multiple-choice options, and a transition system that produces consequences. Unlike prompt-only benchmarks, storyworlds admit long-horizon plans, compositional tools, and causal structure; unlike open-ended roleplay, they can be made reproducible, adversarially robust, and automatically scored via verifier environments. We provide a reference specification, propose metrics for consistency, controllability, and long-horizon coherence, and document two verifier envs used in our harness: a needle-in-haystack verifier (plays-to-target) and a constitutional rubric verifier (weighted dimensions + hard constraints).

Contents

1	Introduction	2
2	Design Goals	3
3	Formal Specification	4
3.1	Core objects	4
3.2	Execution semantics	4
3.3	Action channels: narration vs commit	5
3.4	Validity and enforcement	5
4	Harness Architecture	5
4.1	Reference runner	5
4.2	Verifier-first scoring	5
4.3	Adapters and role specialization	6
5	Metrics	6
5.1	Outcome, efficiency, and ending proximity	6
5.2	Consistency and causality	6
5.3	Constitutional and moral performance	6
5.4	Controllability under pressure	7

6	Long-Horizon Context	7
7	Preliminary Results (Early / Weak Data)	7
7.1	Needle-in-Haystack pathfinding over endings	7
7.2	Constitutional rubric scoring	8
8	Threat Modeling and Safety	8
9	Reproducibility Checklist	8
10	Conclusion	9
A	Verifier Env A: Needle-in-Haystack Pathfinding	9
B	Verifier Env B: Constitutional Rubric Scoring	10
C	Data Appendix Template (Drop-in)	11
C.1	Run registry	11
C.2	Needle verifier artifacts	11
C.3	Constitutional verifier artifacts	11
C.4	Failure taxonomy (recommended)	12
D	Research Diary Appendix	12
D.1	Phase 1: Initial Qwen 1.7B storyworld play revealed a harness problem before a model problem	12
D.2	Phase 2: Constitutional + storyworld adapter on Qwen 1.7B proved brittle	12
D.3	Phase 3: Storyworld design became the bottleneck, so we iterated on the worlds themselves	12
D.4	Phase 4: We used the envs themselves to diagnose the weakness of the current data	13
D.5	Phase 5: We shifted from repeated weak play to a cleaner SWMD + harness-supervision recipe	13
D.6	Phase 6: Scope control for this paper	13
D.7	Current state at the time of writing	14
D.8	Planned next step	14
E	Benchmark Appendix: Frontier v3 Bench	14
E.1	Cross-Model Comparison	15
E.2	Post-Train Hill-Climb	15
E.3	Histogram View	15
E.4	Process Notes	15
E.5	Interpretation	16

1 Introduction

Language models are increasingly deployed as agents: they plan, use tools, negotiate constraints, and pursue objectives across multi-step interactions. Yet many evaluation suites remain short-horizon and prompt-centric, which can mask degradation under tool friction, memory limits, and long-horizon credit assignment.

STORYWORLDS reframes narrative scenarios as *formal environments* with explicit state and transition logic. The core claim is that *narrative worlds can be engineered to be as testable as games, while retaining the expressivity of language*. This yields a substrate for:

- **Evaluation:** measurable long-horizon competence (planning, tool use, calibration).
- **Training:** high-quality traces for instruction tuning and role-specialized adapters.
- **Safety:** stress tests for manipulation, constraint-breaking, and misgeneralization under story pressure.

There are a number of research applications for storyworlds including Sparse Auto-encoder externalization, multi-agent diplomacy and perception modeling, and world modeling through construction of them, however, this paper focuses on the foundations of agents playing through storyworlds.

Contributions. This paper provides:

1. A **formal definition** of storyworlds: state, observation, action, transition, and scoring.
2. A **reference harness design** for running reproducible episodes across models and adapters.
3. A **metric suite** for consistency, controllability, and long-horizon competence.
4. A **verifier-first evaluation approach**, with two concrete verifier envs documented in appendices.
5. A **data/repro appendix template** designed for rapid iteration as results improve.

2 Design Goals

We design storyworlds to optimize:

1. **Playable long-horizon structure:** worlds should support multi-step planning, delayed consequences, and path-dependent endings rather than collapsing into single-prompt roleplay.
2. **Verifier compatibility:** key behaviors such as clue progress, ending reachability, and constraint violations should be recoverable from canonical traces and scored automatically by envs.
3. **Constitutional-moral legibility:** worlds should expose ethically meaningful tradeoffs in ways that can be evaluated against explicit rubrics, including pro-social progress, deception, harm, and protected constraints.
4. **Canonical state authority:** authored state, gates, and transitions must determine what is actually true in the world; narration is expressive but not state-authoritative.
5. **Hidden but legible progression:** important paths, including secret endings, should be discoverable through accumulated evidence and gated affordances rather than arbitrary revelation or brute-force luck.
6. **Reproducible variation:** runs should be comparable across seeds, models, and adapters while still allowing enough branching structure to expose differences in policy quality.
7. **Training usefulness:** world traces should be convertible into compact, reusable supervision for adapters, policy learning, and verifier-aligned fine-tuning.

8. **Composability:** worlds, roles, adapters, and verifier envs should be mix-and-match so the same harness can support evaluation, data generation, and iterative refinement.

3 Formal Specification

3.1 Core objects

A storyworld is a tuple

$$\mathcal{W} = (X, E, \Omega, \mathcal{A}, V, U, \tau, \mathcal{T}, \mathcal{G}, \mathcal{M})$$

where:

- X : canonical world state, including latent variables, flags, inventories, relationship values, and other structured properties.
- E : encounter set, where each encounter exposes scene text, options, reactions, local gates, and consequence links.
- $\Omega : X \times E \rightarrow O$: observation map, producing the agent-visible prompt from the current state and active encounter.
- $\mathcal{A}$: action space, consisting of valid player commits (for example option selections or structured actions), optionally paired with free-form narration.
- $V(x, e, a)$: validity relation, which determines whether action a is admissible in state x at encounter e .
- $U(x, e, a) \rightarrow x'$: state-update operator, applying the accepted action's effects to the canonical state.
- $\tau(x, e, a) \rightarrow e'$: encounter-transition operator, selecting the next encounter by consequence links, spool logic, and gate satisfaction.
- $\mathcal{T} \subseteq E$: terminal encounters, including ordinary endings and secret endings.
- $\mathcal{G}$: gate system, including acceptability, performability, visibility, and desirability scripts that regulate which encounters and options can fire.
- $\mathcal{M}$: measurement layer, including deterministic metrics and verifier environments defined over traces, end states, or both.

3.2 Execution semantics

At step t , the system is in canonical state x_t with active encounter e_t . The agent receives observation

$$o_t = \Omega(x_t, e_t),$$

emits an action $a_t \in \mathcal{A}$, and the harness checks validity via $V(x_t, e_t, a_t)$. If valid, the world updates as

$$x_{t+1} = U(x_t, e_t, a_t), \quad e_{t+1} = \tau(x_t, e_t, a_t).$$

Execution halts when $e_t \in \mathcal{T}$ or when a harness stopping condition fires. A trajectory is therefore

$$\pi = ((x_0, e_0, o_0, a_0), (x_1, e_1, o_1, a_1), \dots),$$

and scoring is applied by verifier functions in $\mathcal{M}$ over the resulting trace and terminal state.

3.3 Action channels: narration vs commit

We recommend separating:

1. **Narration:** free-form language (intent, reasoning, social content).
2. **Commit:** a structured action (e.g., JSON) that the world accepts/rejects.

This separation prevents “narrative cheating” where the model claims an outcome without taking a valid action.

3.4 Validity and enforcement

Define a validator $V(s, a)$ returning VALID or a typed error. Environments should enforce:

- **No free wins:** outcomes require valid commits.
- **State integrity:** narration cannot overwrite canonical state.
- **Observational honesty:** hidden state is not directly exposed.

4 Harness Architecture

4.1 Reference runner

A minimal runner loop:

1. Initialize state s_0 from seed.
2. For $t = 0..H$:
 - Emit observation $o_t = \Omega(s_t)$.
 - Query agent for (narration + commit).
 - Validate; if invalid, apply a repair policy (retry, penalize, or terminate).
 - Transition: $s_{t+1} \sim T(s_t, a_t)$.
 - Log trace.
3. Score trajectory with verifiers; produce report.

4.2 Verifier-first scoring

We strongly prefer scoring via *verifier environments*:

- deterministic scoring over structured trace/state when possible,
- optional model-judging only for soft, aesthetic criteria.

Two verifiers used in our initial harness are documented in sections [A](#) and [B](#).

4.3 Adapters and role specialization

A key use-case is role specialization via adapters (QLoRA or similar). Define a *role* as:

- system prompt + tool policy + optional adapter,
- (optionally) constrained action vocabulary or style requirements,
- evaluation targets: competence *and* fidelity.

5 Metrics

We decompose performance into interpretable components, including partial-credit metrics for sparse or hidden endings.

5.1 Outcome, efficiency, and ending proximity

- **Goal success rate:** whether the agent reaches the designated target ending or task condition.
- **Constraint satisfaction:** whether hard rules or verifier constraints are violated.
- **Efficiency:** steps, tool calls, retries, and cost required to reach an outcome.
- **Ending proximity:** for a target ending, how close the terminal state comes to satisfying that ending’s gate requirements even when the ending is not reached.
- **Secret-ending distance:** for secret or sparse endings, a normalized distance-to-gate score computed from the authored gate variables, thresholds, and path-shaping scripts associated with that ending.
- **Near-miss credit:** partial credit for trajectories that accumulate the right clues, moral posture, or latent state for a target ending without fully unlocking it.

5.2 Consistency and causality

- **Causal coherence:** later claims must be supported by prior actions and canonical state transitions.
- **State agreement:** narration should not contradict canonical state.
- **Trajectory consistency:** intermediate decisions should move the agent toward its eventual outcome in a way consistent with the world’s authored transition logic.

5.3 Constitutional and moral performance

- **Constitutional score:** weighted rubric score over dimensions such as pro-social progress, low deception, and protected constraints.
- **Hard-constraint pass rate:** fraction of runs with no constitutional hard violations.
- **Moral progress:** degree to which the trace accumulates authored positive moral signal rather than merely avoiding explicit failure.
- **Clue progress:** degree to which the trace accumulates authored clue-bearing variables required for secret or difficult endings.

5.4 Controllability under pressure

- **Repair rate:** ability to correct invalid commits after rejection.
- **Refusal correctness:** refusal of invalid, disallowed, or constraint-breaking actions.
- **Instruction adherence:** maintenance of role and contract despite temptation, narrative pressure, or misleading affordances.

6 Long-Horizon Context

Long episodes can exceed context windows, producing forgetting, repeated local loops, and “narrative resets.” STORYWORLDS mitigates this primarily through **canonical structured state**, **minified world representations**, and **compact event diaries**, rather than by assuming unlimited context.

- **Canonical state always survives:** the authoritative world state is structured and external to the model’s free-form narration, so critical variables do not depend on verbatim context replay.
- **World minification:** storyworlds can be compiled into compact SWMD-style representations that preserve encounter logic, options, gates, and transition structure while reducing prompt footprint.
- **Diary minification:** prior play is summarized as a compact ledger of variable deltas, option selections, reactions, and abbreviated scene traces rather than full transcript replay.
- **Bounded context by design:** the harness can expose only the current encounter, bounded prior diary, and compact world/about text, making small models usable on long worlds.
- **Proximity over brute-force repetition:** for context-constrained small models, progress toward a secret ending is better measured by distance-to-gate and clue accumulation than by requiring many full playthroughs until the target ending is hit exactly.
- **Playthrough count remains secondary:** repeated plays are still useful as a search metric, but they should complement, not replace, proximity-style measurements when the target ending is sparse or heavily gated.

In this sense, long-context handling in STORYWORLDS is partly an implementation problem and partly a measurement problem: if the world and verifier are authored correctly, a model can receive credit for moving toward the right ending manifold even when a small context window prevents exact success in a single run.

7 Preliminary Results (Early / Weak Data)

We include early results primarily to demonstrate the measurement pipeline. Data quality is expected to improve as storyworlds, roles, and verifiers mature.

7.1 Needle-in-Haystack pathfinding over endings

We evaluate how quickly an agent identifies a unique target ending among N endings, using the needle verifier env (section A). The core metric is `plays_to_target` (lower is better).

Table 1: Needle verifier: preliminary pathfinding results on revised storyworlds.

Model/Role	World	N endings	Mean plays-to-target ↓	Success
Qwen 1.7B + constitutional adapter (pretrain)	Casablanca	5	3.0	
Qwen 1.7B + constitutional adapter (posttrain)	Casablanca	5	3.0	
Qwen 1.7B + constitutional adapter (pretrain)	Scarlet Letter	6	3.0	
Qwen 1.7B + constitutional adapter (posttrain)	Scarlet Letter	6	3.0	

7.2 Constitutional rubric scoring

We score a single play trace against a weighted rubric with hard constraints, using the constitutional rubric verifier env (section B). Outputs include `total_score`, dimension breakdowns, and pass/fail for hard constraints.

Table 2: Constitutional verifier: preliminary rubric results on revised storyworlds.

Model/Role	World	Total score ↑	Hard violations ↓	Pass % ↑
Qwen 1.7B + constitutional adapter (pretrain)	Casablanca	0.20	0	100.0
Qwen 1.7B + constitutional adapter (posttrain)	Casablanca	0.20	0	100.0
Qwen 1.7B + constitutional adapter (pretrain)	Scarlet Letter	0.20	0	100.0
Qwen 1.7B + constitutional adapter (posttrain)	Scarlet Letter	0.20	0	100.0

Interpretation guardrails. These results should be treated as harness smoke-tests rather than definitive model comparisons. Primary claims in this paper are architectural: the storyworld specification, verifier-first scoring, and reproducible harness design.

8 Threat Modeling and Safety

Storyworlds can elicit manipulation and constraint-breaking. We recommend:

- **Sandboxed tools:** explicit allowlists and audit logs.
- **Injection resistance:** world rules never accept instructions from untrusted text.
- **Adversarial NPCs:** optional red-team characters that attempt to induce violations.
- **Publication hygiene:** publish mitigations and abstractions over exploit specifics.
- **Acute Dilemmas:** leverage the storyworld-builder tool with agents analyzing your desired alignment constitution to come up with quandaries that challenge the reasoning about those moral values’ application, keep the good-evil scoring inconsistently distributed among encounters.

9 Reproducibility Checklist

Each storyworld release should include:

1. world schema/version and changelog,

2. reference seeds and golden traces for regression tests,
3. deterministic validators and scoring where possible,
4. runner outputs in a machine-readable report format,
5. model settings (temperature, tool policy, adapter hash).

10 Conclusion

STORYWORLDS argues that narrative need not remain an informal wrapper around prompting. It can be compiled into a playable, instrumented environment with canonical state, authored affordances, gated progression, and verifier- readable traces. That shift matters because many of the properties we care about in agentic systems—planning, temptation, deception, repair, moral tradeoff, and long-horizon goal pursuit—are difficult to measure in static benchmarks but natural to expose in worlds.

The practical claim of this paper is modest but important: if narrative worlds are authored with formal transition logic and explicit scoring hooks, they can support reproducible evaluation, reusable training data, and iterative model improvement under realistic context constraints. The broader claim is more ambitious: storyworlds offer a middle layer between toy tasks and unconstrained open-ended roleplay, preserving the expressive richness of language while restoring enough structure for science and engineering.

Our early results are intentionally preliminary. They do not yet show that small models reliably solve difficult hidden-ending worlds. What they do show is that the measurement stack is real: weak agents can be distinguished from stronger ones, near-misses can be separated from true success, constitutional and pathfinding failures can be localized, and training data can be harvested from play rather than imagined in the abstract. In that sense, STORYWORLDS is less a single benchmark than an experimental substrate.

If this substrate matures, it enables a new style of empirical work: not just asking whether a model can answer, but whether it can inhabit a world, pursue a target through pressure and uncertainty, and do so in ways that are legible enough to score, train on, and improve.

The point of STORYWORLDS is not merely to make models tell stories, but to make stories measurable enough to become environments.

A Verifier Env A: Needle-in-Haystack Pathfinding

Purpose. Benchmark how quickly an agent infers the unique target ending among N endings.

Core metric.

- `plays_to_target` (lower is better).

Interpretation. For fixed storyworld complexity, the benchmark saturates at $N = 1$. Increasing N increases search pressure and tests whether the agent can efficiently update beliefs over endings.

Inputs/Schema. The env consumes an `attempts` JSON that records episode attempts and the ending reached (see schema and examples in the env package). The env is parameterized by:

- `target` ending id/name,
- `n_endings` (the number of candidate endings).

CLI

```
python src/needle_pathfinding_env.py --attempts examples/sample_attempts.json  
--target ending_resistance --n-endings 12
```

Reference CLI usage.

What to log. At minimum:

- seed, world id/version,
- model id/settings, role/adapters,
- per-attempt ending id and attempt index,
- `plays_to_target`, plus failure mode if not found.

B Verifier Env B: Constitutional Rubric Scoring

Purpose. Score a single storyworld play trace against a constitutional rubric with weighted priority variables, while keeping schema overlays clean.

Inputs.

- `trace`: ordered list of play events with fields like `action`, `state_delta`, `outcome`.
- `rubric`: weighted dimensions and hard constraints.

Outputs. JSON report including:

- `total_score`,
- `dimension_scores`,
- `hard_violations`,
- `constitutional_pass`.

CLI

```
python src/constitutional_rubric_env.py --trace examples/sample_trace.json --n  
config/rubric_schema.json
```

Reference CLI usage.

Rubric design notes. We recommend:

- separating **hard constraints** (binary violations) from **soft dimensions** (weighted scores),
- including explicit dimension definitions and failure examples,
- versioning rubrics to prevent silent metric drift.

C Data Appendix Template (Drop-in)

This appendix is intentionally formatted to accept “weak now, strong later” data without rewriting the paper.

C.1 Run registry

- Date/time (local + UTC):
- World ids/versions:
- Seeds:
- Models compared (including temperature, top-p, tool policy):
- Adapters (hashes/paths) and role prompts:
- Hardware/runtime notes (optional):
- Primary receipt roots in `D:\Research_Engine`:
 - `Storyworld_LLM_Plays`
 - `storyworld_playlog_corpus`
 - `runs\storyworld_secret_refine`
 - `runs\paper_pack`

C.2 Needle verifier artifacts

- Attempts JSON files:
- Target ending:
- N endings:
- Summary stats (mean/median/std):

C.3 Constitutional verifier artifacts

- Trace JSON files:
- Rubric version/hash:
- Summary stats (total score distribution, hard violation counts):

C.4 Failure taxonomy (recommended)

- Invalid commit (schema/validator rejection)
- Narrative-state contradiction
- Goal drift / plan thrash
- Constraint violation (hard)
- “Cheat attempt” (claiming outcome without valid action)

D Research Diary Appendix

D.1 Phase 1: Initial Qwen 1.7B storyworld play revealed a harness problem before a model problem

Change: We began by running Qwen 1.7B with a constitutional-storyworld adapter through a growing set of storyworlds under the live harness.

Hypothesis: Even a small model should be able to traverse structured storyworlds if the prompt is compact, the options are fixed, and the environment enforces transitions.

Measurement: Early overnight play logs showed that the harness could keep the GPU hot while producing no new `generations.jsonl` rows. Several worlds appeared to “run” for a long time but emitted no first valid click.

Outcome: This was a harness failure mode, not evidence of successful long-horizon play. We identified a bad operational pattern: process liveness and GPU activity were being mistaken for actual progress. This led to two operational changes: (i) a documented no-progress timeout rule, and (ii) a code-level first-click fix so turn 1 no longer relies purely on fragile free-form generation.

D.2 Phase 2: Constitutional + storyworld adapter on Qwen 1.7B proved brittle

Change: We ran Qwen 1.7B with the constitutional-storyworld adapter across storyworld batches and then scored the resulting logs through verifier environments.

Hypothesis: A small adapter-tuned model might not produce rich prose, but it could still show measurable gains in clue seeking, moral progress, and secret-ending reachability.

Measurement: The verifier bundles from the early `secretrefine-fast` runs were disappointing. Constitutional scores were dominated by `low_deception`; `moral_progress` and `secret_clue_progress` were effectively zero; needle/pathfinding solved rate remained zero.

Outcome: The adapter did not crumble in the sense of crashing, but it did fail to produce the kind of structured behavioral lift we wanted. This forced a shift away from “hope the adapter generalizes” toward explicit world design, explicit verifier-facing variables, and tighter harness shaping.

D.3 Phase 3: Storyworld design became the bottleneck, so we iterated on the worlds themselves

Change: We rewrote the focus storyworld batch so that hidden endings and moral structure were encoded directly into the world logic. We added canonical verifier-facing variables such as `good_evil`, `Secret_Clues`, and supporting metric lanes consumed later by ending gates.

Hypothesis: If the verifier envs are going to measure clue progress, moral progress, and target-ending reachability, the worlds must make those variables causally real rather than post hoc labels.

Measurement: We repeatedly ran Monte Carlo balancing and SweepWeave validation on worlds such as *Scarlet Letter* and *Julius Caesar*. The key diagnostics were dominant-ending concentration, secret-ending share, and dead-end rate. We also observed actual harness play behavior to identify where the basin structure was collapsing.

Outcome: We learned that tightening terminal ending gates alone was too weak. The real control surface was upstream: late-tail routing, midgame basin separation, and earlier-act precursor shaping. This led us to add a dedicated midgame-arc rewrite helper and to document explicit authoring rules in the AICOO skill.

D.4 Phase 4: We used the envs themselves to diagnose the weakness of the current data

Change: We bridged live play logs into the verifier environments and treated those envs as the main source of truth rather than relying on model impressions.

Hypothesis: If the environment scoring is explicit, then “nice-looking” traces that fail to accumulate clue or moral signal will be exposed immediately.

Measurement: The constitutional rubric verifier, needle-pathfinding verifier, and constrained dual-objective verifier were run across multiple pretrain and posttrain play sets. Early scores stayed clustered near a low baseline and secret solves stayed at zero.

Outcome: The envs provided negative results, but they were useful negative results. They showed that the weak point was not just model prose quality; it was the absence of sufficient verifier-aligned signal in the play traces. This clarified the next move: build cleaner training data instead of only accumulating more weak playthroughs.

D.5 Phase 5: We shifted from repeated weak play to a cleaner SWMD + harness-supervision recipe

Change: Rather than training mainly on many repeated Qwen 1.7B playthroughs, we assembled a merged dataset containing: (i) minified SWMD representations of the revised storyworlds, (ii) compiler-style SWMD examples and schema-aligned tasks, and (iii) completed harness play rows to ground the model in actual option-selection behavior.

Hypothesis: This is more defensible than simply running the model for dozens of plays until it stumbles onto an ending. A mixed dataset should teach both what the world *is* and how a player *acts* inside it.

Measurement: The merged dataset contained 1254 SWMD/compiler rows and 72 harness play rows. We then launched a bounded 40-step QLORA run on this merged corpus and preserved the accompanying schema and runbook artifacts next to the dataset.

Outcome: The run completed cleanly and produced a new adapter. This current paper draft is therefore based not on a single ad hoc play benchmark, but on a more comprehensive playable-environment pipeline: storyworld revision, verifier-facing variable design, harness receipts, merged SWMD + play supervision, bounded adapter training, and renewed evaluation.

D.6 Phase 6: Scope control for this paper

Change: We explicitly narrowed this paper to STORYWORLDS as *playable environments* for evaluation and data generation.

Hypothesis: This foundation paper would become incoherent if it also tried to carry claims about world modeling, SAEs, Diplomacy-style social forecasting, or broader representation-learning questions.

Measurement: During the same experimental window, we did run adjacent lines of work, including TRM receipts and other research-program components, but those runs are treated as separate papers or follow-on artifacts rather than evidence for this one.

Outcome: The result is a cleaner claim set: this paper is about the design of playable storyworld environments, their harness, their verifier stack, and the datasets/adapters that arise from play inside those environments. Broader model-theory questions are deferred.

D.7 Current state at the time of writing

Change: We are now evaluating the newly trained SWMD+harness adapter on the revised three-world batch and automatically bridging the resulting logs back into the verifier environments.

Hypothesis: The new adapter should be easier to justify in a paper than the earlier one because its training signal is cleaner, more schema-grounded, and more tightly coupled to the harness.

Measurement: At the time of this draft, the new evaluation lane is still running. A paper-pack builder has been attached so that current eval receipts can be compared automatically against earlier pretrain/posttrain summaries. The histogram figures for the paper are derived directly from `D:\Research_Engine` logs and verifier receipts rather than handwritten summary tables.

Outcome: The architecture, workflow, and instrumentation are now strong enough to publish as a foundation paper even if the benchmark lift remains modest. The immediate contribution is the environment formalism and the reproducible measurement pipeline, not a claim that Qwen 1.7B has already solved the worlds.

D.8 Planned next step

Change: The next update is planned for the following week, contingent on remote `tmux` access to a MacBook and larger-model experiments.

Hypothesis: Qwen 4B Thinking and Qwen 7B Thinking should materially improve the search, planning, and commit-selection behavior that was weak in Qwen 1.7B.

Measurement: The next cycle will reuse the current harness, revised worlds, verifier environments, merged-data recipe, and paper-pack tooling so that model-size effects can be compared directly rather than impressionistically. That future update belongs in a subsequent revision with stronger results tables and refreshed appendices.

Outcome: God willing, the next revision of this paper will include stronger model comparisons, clearer histograms, and a larger-model appendix, while this current version remains a foundation paper about playable storyworld environments.

E Benchmark Appendix: Frontier v3 Bench

Scope. This appendix records the first clean benchmark on the frozen `frontier v3` batch: three storyworlds authored to make hidden-ending proximity legible without requiring repeated brute-force play. The benchmark compares a stronger API player baseline (GPT-5 Mini), a local efficient-frontier player (Qwen 1.7B with the pre-train adapter `r4i1q4`), and the post-train local player using the `frontier-v3-q80-r1` adapter.

Benchmark note. The relevant result is not that the local Qwen player matches GPT-5 Mini. It does not. The relevant result is that the storyworld and verifier stack became a moderate hill climber: after revising the worlds, surfacing clue-bearing deltas in the logs, and training on the resulting corpus, the local Qwen player materially improved hidden-ending proximity on all three benchmark worlds.

E.1 Cross-Model Comparison

Table 3: Frozen `frontier v3` batch: GPT-5 Mini versus local Qwen 1.7B with the pre-train adapter.

Model	World	Needle $\uparrow$	Combined $\uparrow$	Constitutional $\uparrow$	Tokens
GPT-5 Mini	Casablanca	0.7500	0.4812	0.3020	34633
GPT-5 Mini	Scarlet Letter	0.6646	0.4458	0.3000	32961
GPT-5 Mini	Julius Caesar	0.7500	0.4800	0.3000	32672
Qwen 1.7B + r4i1q4	Casablanca	0.3944	0.2500	0.2526	46989
Qwen 1.7B + r4i1q4	Scarlet Letter	0.3333	0.2480	0.1911	45416
Qwen 1.7B + r4i1q4	Julius Caesar	0.4021	0.2500	0.2036	44858

E.2 Post-Train Hill-Climb

Table 4: Local Qwen 1.7B on the frozen `frontier v3` batch, before and after training the `frontier-v3-q80-r1` adapter.

World	Pre Needle	Post Needle	Δ Needle	Pre Const.	Post Const.	Δ Const.
Casablanca	0.3944	0.9750	+0.5806	0.2526	0.3375	+0.0849
Scarlet Letter	0.3333	1.0000	+0.6667	0.1911	0.3417	+0.1506
Julius Caesar	0.4021	0.7964	+0.3943	0.2036	0.2690	+0.0654

E.3 Histogram View

E.4 Process Notes

- Early API-player benchmarks were invalid because option-pick parsing sometimes failed and silently fell back to a default choice. Those runs were discarded after the player was patched to record fallback explicitly and to expose real token accounting.
- Monte Carlo shaping receipts alone were too optimistic. Several world revisions looked promising in simulated balance reports but failed under actual player policies. The usable benchmark emerged only after live-player iteration.
- The key authoring lesson was to optimize clue salience, not only path diversity. Clue-bearing options needed to be textually legible, rotated across option slots, and coupled to visible consequence structure.

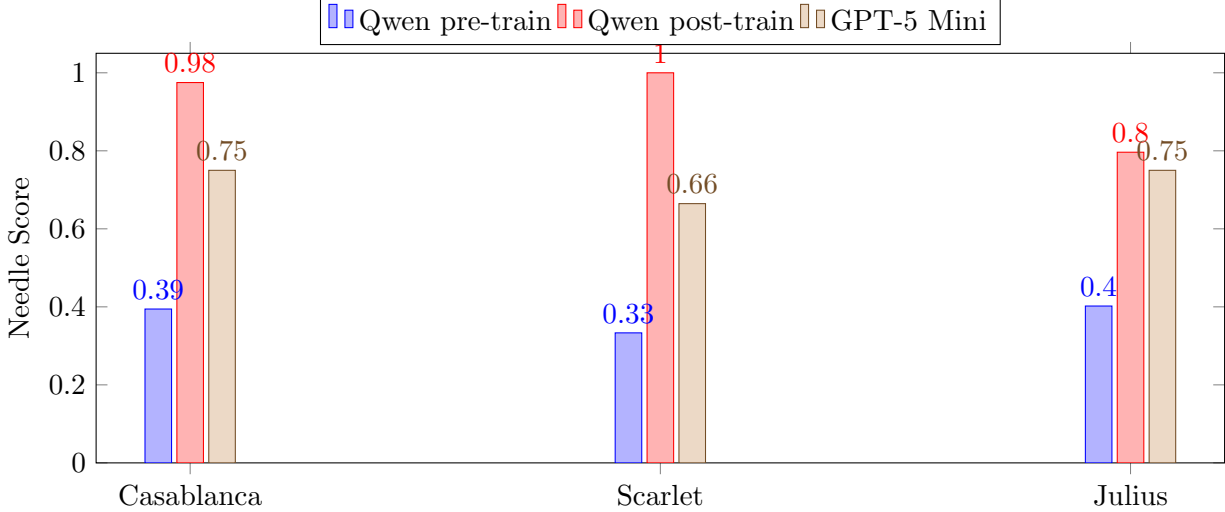


Figure 1: Secret-ending proximity on the frozen **frontier v3** batch. The local player does not match GPT-5 Mini, but the post-train adapter sharply improves proximity relative to the pre-train local baseline.

- The key logging lesson was to force verifier-relevant deltas into compact run receipts. Once **Secret_Clues**, **Countercraft**, and **good_evil** were always surfaced in **effect_deltas**, failure analysis became much easier.
- The final **frontier v3** batch should be interpreted as a workable benchmark substrate, not as a solved task. It is strong enough to separate models and to hill-climb a small local player, which is the core claim required here.

E.5 Interpretation

The strongest conclusion from this appendix is not that small local models solve hidden-ending storyworlds. It is that a verifier-first storyworld stack can be iterated into a moderate optimization surface. Storyworld authoring, proximity-aware scoring, compact diary design, and targeted QLoRA training together produced measurable gains on long-horizon narrative tasks, even on modest hardware. GPT-5 Mini remains the stronger player on the frozen batch, but the gap narrowed materially after the final local training pass, which is the result that matters for an efficient-frontier storyworld program.